

When you edit a previous message in a modern chat interface (like modifying Turn 3 out of 5), **the past turns are neither permanently deleted nor do they get passed to the LLM as active context.**

Instead, chat platforms handle edits using a **Branching Tree Structure** (similar to Git commit history).

## 1. How the Database Handles It (Does it delete?)

No, standard production chat applications **do not delete** the old turns. Deleting them would destroy your ability to switch back to your original path later.

Instead, databases (like PostgreSQL, SQLite, or MongoDB) store messages in a **relational tree** using parent-child pointers:

- **The Flat Illusion:** To the user, a chat looks like a linear list (Turn 1 -> Turn 2 -> Turn 3 -> Turn 4 -> Turn 5).
- 
- **The Reality (Message Nodes):** Every single message has a unique `id` and a `parent_id`. When you edit Turn 3, the system **keeps the old Turn 3 and Turn 4 nodes in the database**, but creates a *brand new* branch pointing off Turn 2.
- 

Example Database Schema (`messages` table):

| id          | parent_id  | role      | content                        |
|-------------|------------|-----------|--------------------------------|
| msg-1       | null       | user      | "Hello"                        |
| msg-2       | msg-1      | assistant | "Hi there!"                    |
| msg-3 (Old) | msg-2      | user      | "What is 2+2?"                 |
| msg-4 (Old) | msg-3      | assistant | "2+2 is 4."                    |
| msg-3-edit  | msg-2      | user      | "What is 3+3?" (The Edit)      |
| msg-5       | msg-3-edit | assistant | "3+3 is 6." (The New Response) |

Both branches live safely side-by-side in the database. When you click the `< 1/2 >` arrows on an edited message in a UI, you are simply telling the frontend to switch which path of `parent_id` links it is rendering.

## 2. How the Context Passed to the LLM Works

When you submit that edit on Turn 3, the backend does **not** send the old Turns 3, 4, or 5 to the LLM.

The orchestration layer compiles the context array by **walking backward from the active leaf node** (the newest message on your active branch):

1. It starts at your newly edited Turn 3.
- 2.
3. It looks at its `parent_id` and finds Turn 2.
- 4.
5. It looks at Turn 2's `parent_id` and finds Turn 1.
- 6.
7. It stops when it hits `parent_id: null`.
- 8.

**What gets sent to the LLM API payload:**

JSON

```
"role" "user" "content" "Turn 1"
"role" "assistant" "content" "Turn 2"
"role" "user" "content" "Turn 3 (The Edited Version)"
```

The old responses (Turns 4 and 5) are completely cut out of the array sent to the model. To the LLM, they never happened.

## Open-Source Projects to Explore

If you want to look at how this tree-based branching and database architecture is written in code, check out these repositories:

- [open-webui / open-webui \(GitHub\)](#): Manages chat histories via a dictionary of nodes containing `parentId` and `childrenIds` arrays. Its documentation and codebase explicitly detail how history pointers switch active branches upon message editing.
- 
- [danny-avila / LibreChat \(GitHub\)](#): Implements a relational database schema where the `messages` table uses self-referencing foreign keys (`parent_id`) to map out complex branching dialogue trees rather than linear lists.